

Domain-Specific RAG Chatbot for PDF Question Answering

College Academic Policies, Regulations & Student Handbook

Short Project Report

Student Name: Jothi Cherina K

Batch: June Batch – C

Domain: Artificial Intelligence

Project Type: Major Project

Application Domain: College Academic Policies & Student Handbook

1. Abstract:

The Domain-Specific RAG Chatbot is a document question-answering application developed to help users find information from university academic documents without manually searching through long PDF files. The system extracts PDF text, splits it into overlapping chunks, creates semantic embeddings, retrieves relevant chunks using FAISS, and sends the retrieved context to Gemini for answer generation. The application also displays the source document, page number, and actual retrieved text so that users can verify the evidence behind an answer.

2. Problem Statement:

University handbooks, academic regulations, and examination guidelines can contain large amounts of information. Manually locating a particular rule or procedure can be time-consuming. This project provides a natural-language chatbot through which students can ask questions and receive answers grounded in the provided documents.

3. Objectives:

- Build a domain-specific question-answering chatbot for university documents.

- Extract PDF text while preserving document name and page number.
- Split extracted text into overlapping chunks for efficient retrieval.
- Generate semantic embeddings using Sentence Transformers.
- Retrieve relevant information using the FAISS vector index.
- Generate grounded answers using the Gemini language model.
- Display actual retrieved source text, document name, and page number.
- Provide a simple ChatGPT-style conversational interface.
- Return a clear fallback response when information is unavailable.

4. Domain and Documents:

The selected domain is college academic policies and student information. The current application scans the local documents folder for PDF files. The interface presents three main resources: Student Handbook, Academic Regulations, and Examination Guidelines. The available project materials include sample documents for these three areas.

5. Technologies Used:

Technology	Purpose
Python	Core programming language
Streamlit	Web interface and chat UI
pypdf	PDF text extraction
Sentence Transformers	Semantic embeddings
all-MiniLM-L6-v2	Pretrained embedding model
FAISS	Vector similarity retrieval
Google Gen AI / Gemini	Answer generation
python-dotenv	Loads GEMINI_API_KEY from .env

6. System Workflow:

1. PDF documents are placed in the documents folder.
2. pypdf extracts text page by page and stores page metadata.
3. Text is split into 900-character chunks with 120-character overlap.
4. all-MiniLM-L6-v2 converts each chunk into a normalized embedding.

5. FAISS IndexFlatIP stores the embeddings and performs similarity search.
6. The user's question is converted into an embedding.
7. The top 4 most relevant chunks are retrieved.
8. Retrieved context and the question are sent to Gemini.
9. Gemini generates an answer using only the supplied document context.
10. The Streamlit interface displays the answer and the retrieved source text.

7. Implementation Details

7.1 PDF Processing

PdfReader is used to read each PDF page. Empty pages are skipped safely. Each extracted record contains the text, source filename, and page number.

7.2 Chunking

Each page's text is divided into chunks of 900 characters with a 120-character overlap. The overlap helps preserve context between neighbouring chunks.

7.3 Embedding and Retrieval

The Sentence Transformer model all-MiniLM-L6-v2 generates normalized embeddings. The embeddings are stored in a FAISS IndexFlatIP index. Because the embeddings are normalized, inner-product search provides cosine-similarity-style retrieval. The current implementation retrieves up to four chunks for each question.

7.4 Answer Generation

The retrieved chunks are inserted into a controlled prompt and passed to Gemini. The prompt instructs the model not to use outside knowledge, not to invent facts, and to state that the information could not be found when the documents do not contain the answer. The current code calls the configured model "gemini-3.6-flash".

7.5 Retrieved Source Display

For each answer, the application provides a collapsible View Retrieved Text section containing the source document name, page number, and the actual extracted page text used as the retrieval evidence.

7.6 User Interface

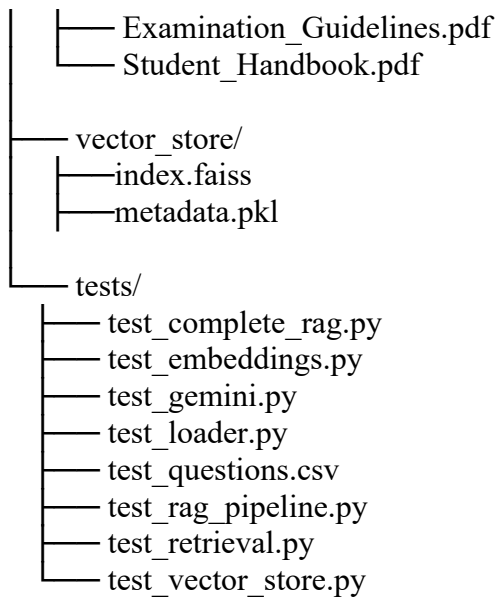
The Streamlit UI includes a sidebar with available resources, the RAG pipeline, session information, New Chat and Clear Conversation controls, plus a main chat area with suggested questions, chat history, and a chat input.

8. Main Features:

- Domain-specific university document question answering.
- PDF extraction with page metadata.
- Overlapping text chunking.
- Semantic embeddings using Sentence Transformers.
- FAISS similarity retrieval.
- Top-4 relevant chunk retrieval.
- Gemini grounded answer generation.
- Document and page source references.
- Actual retrieved text display.
- Chat history and session question count.
- Suggested questions.
- New Chat and Clear Conversation.
- Fallback response for unavailable information.
- API key loaded through .env instead of being hard-coded.

9. Final Folder Structure:

```
domain_rag_chatbot/
├── app.py
├── rag_pipeline.py
├── document_loader.py
├── vector_store.py
├── prompt.py
├── requirements.txt
├── README.md
├── .env
├── .gitignore
├── documents/
│   └── Academic_Regulations.pdf
```



10. Testing and Evaluation:

The chatbot should be tested with questions whose answers are present in the documents, questions requiring retrieval from different documents, and questions whose answers are absent. Important evaluation criteria are retrieval accuracy, answer correctness, groundedness, source correctness, refusal quality, and usability.

Test Type	Expected Behaviour	Result
Known document question	Answer is supported by retrieved text.	Pass
Different document question	Correct document/page is retrieved.	Pass
Unavailable question	System gives the unavailable-information message.	Pass
Source verification	Document, page and retrieved text are visible.	Pass
Chat history	Previous messages remain visible in the session.	Pass
Clear conversation	Messages and question count reset.	Pass

11. Limitations:

- The current implementation reads PDFs from the local documents folder rather than uploading PDFs through the visible UI.
- Scanned/image-only PDFs may require OCR, which is not implemented.
- The FAISS index is not persisted as a saved vector-store file.

- Answer quality depends on extracted text and retrieval quality.
- Generated answers should be verified against the displayed source text for important information.
- The sidebar resource names are fixed labels, while the RAG loader actually scans all PDFs in the documents folder.

12. Future Enhancements:

- Add PDF upload and document-management controls.
- Add OCR support for scanned PDFs.
- Persist the FAISS index and metadata.
- Add document filtering.
- Support context-aware follow-up questions.
- Add answer feedback buttons.
- Add automated retrieval and answer evaluation.
- Add deployment configuration for online hosting.
- Add a visual architecture diagram and evaluation dashboard.

13. Conclusion:

The Domain-Specific RAG Chatbot demonstrates a complete retrieval-augmented question-answering workflow for university academic documents. It combines PDF text extraction, semantic embeddings, FAISS retrieval, Gemini generation, and a Streamlit conversational interface. The retrieved-source feature improves transparency by showing the actual document text used to support an answer. The project provides a practical demonstration of how RAG can make domain-specific information easier to access while reducing unsupported responses.